Solana Program Rewards

Security Assessment

April 24th, 2026 — Prepared by OtterSec

Akash Gurugunti sud0u53r.ak@osec.io

Gabriel Ottoboni ottoboni@osec.io

Table of Contents

Appendices

01 — Executive Summary

Overview

Solana engaged OtterSec to assess the `rewards` program. This assessment was conducted between April 7th and April 14th, 2026. For more information on our auditing methodology, refer to Appendix B.

Key Findings

We produced 8 findings throughout this audit engagement.

In particular, we identified a vulnerability where the `CloseDirectRecipient` depends on the parent distribution account still being program-owned, so once `CloseDirectDistribution` deletes that PDA, any leftover recipient accounts may no longer be closed, permanently locking the rent in fully claimed `DirectRecipient` accounts (OS-SPR-ADV-03). Additionally, the rewards program prefixes CPI events like Anchor events, but uses a 1-byte custom event discriminator instead of Anchor's required 8-byte hash-based discriminator (OS-SPR-ADV-04). Furthermore, when adding a direct recipient, the function records and allocates the full requested amount, but if the reward mint charges transfer fees, the distribution vault may be underfunded (OS-SPR-ADV-02).

We also suggested updating the claimed amount to include any vested amount paid during NonVested revocation before finalizing the revocation (OS-SPR-SUG-00). We further advised validating the instruction data uniformly in every processor after parsing (OS-SPR-SUG-01).

02 — Scope

The source code was delivered to us in a Git repository at https://github.com/solana-program/rewards. This audit was performed against commit d795849.

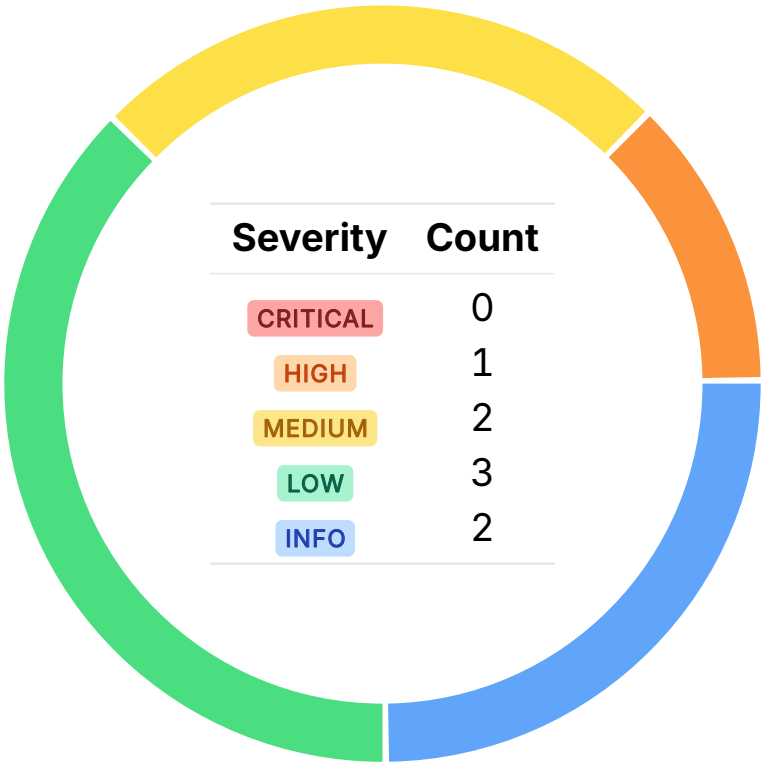
A brief description of the program is as follows:

Name	Description
rewards	A Solana module for distributing rewards through direct allocations, Merkle claims, continuous balance-based pools, or non-transferable points. It handles vesting, claiming, revocation, and works with both SPL Token and Token-2022.

03 — Findings

Overall, we reported 8 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



Severity	Count
CRITICAL	0
HIGH	1
MEDIUM	2
LOW	3
INFO	2

04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in Appendix A.

ID	Severity	Status	Description
OS-SPR-ADV-00	HIGH	RESOLVED	In the revocation paths for Direct and Merkle distributions, the program verifies only that `recipient_token_account` is a valid token-program account, but does not ensure it is controlled by the intended recipient.
OS-SPR-ADV-01	MEDIUM	RESOLVED	On closing `DirectDistribution` its PDA is deleted, enabling the same PDA to be later recreated at the same address, while stale `DirectRecipient` accounts from the old distribution may still exist and still reference that address, resulting in stale recipients from a previous campaign to become valid again and claim against a newly initialized distribution.
OS-SPR-ADV-02	MEDIUM	RESOLVED	`AddDirectRecipient` records and allocates the full requested amount, but if the reward mint charges transfer fees, `TransferChecked` may deposit less than that into the distribution vault, leaving the distribution underfunded on chain.
OS-SPR-ADV-03	LOW	RESOLVED	`CloseDirectRecipient` depends on the parent distribution account still being program-owned, so once `CloseDirectDistribution` deletes that PDA, any leftover recipient accounts may no longer be closed, permanently locking the rent in fully claimed `DirectRecipient` accounts.

OS-SPR-ADV-04	LOW	RESOLVED	The rewards program prefixes CPI events like Anchor events, but uses a 1-byte custom event discriminator instead of Anchor's required 8-byte hash-based discriminator.
OS-SPR-ADV-05	LOW	RESOLVED	`CloseDirectRecipient` requires the `recipient` to authorize closure, but the recovered rent is paid to the original payer, so the caller does not benefit from acting. This misaligned incentive may leave fully claimed recipient accounts unclosed.

Absence of Recipient Token Account Ownership Check HIGH OS-SPR-ADV-00

Description

Both `RevokeDirectClaim` (shown below) and `RevokeMerkleClaim` validate that `recipient_token_account` is owned by the token program, but they do not verify that the token account's owner is actually the intended recipient/claimant. If the instruction pays out vested tokens during `NonVested` revocation, the authority may supply an arbitrary token account for the same mint and redirect those tokens away from the user, thereby stealing the recipient's vested tokens.

>_ *program/src/instructions/direct/revoke_recipient/accounts.rs* RUST

```rust
impl<'a> TryFrom<&'a [AccountView]> for RevokeDirectRecipientAccounts<'a> {
    type Error = ProgramError;

    #[inline(always)]
    fn try_from(accounts: &'a [AccountView]) -> Result<Self, Self::Error> {
        let [authority, distribution, recipient_account, recipient, original_payer, mint,
            ↪  distribution_vault, recipient_token_account, authority_token_account,
            ↪  token_program, event_authority, program] =
           accounts
        else {
            return Err(ProgramError::NotEnoughAccountKeys);
        };
        [...]
        // 5. Validate token account ownership
        verify_owned_by(mint, token_program.address())?;
        verify_owned_by(recipient_token_account, token_program.address())?;
        verify_owned_by(authority_token_account, token_program.address())?;
        [...]
    }
}
```

Remediation

Validate the token account's (`recipient_token_account`) `owner` field matches the recipient/claimant in `RevokeDirectClaim` and `RevokeMerkleClaim`.

Patch

Resolved in PR#33.

Recreated Distribution PDA Revives Stale Recipients MEDIUM OS-SPR-ADV-01

Description

Closing a `DirectDistribution` fully deletes its PDA, but the same PDA may later be recreated because its address is deterministically derived from the same seeds. If old `DirectRecipient` accounts were never closed, they still reference that distribution address and may become valid again after re-creation. This may enable stale recipients to claim from a newly funded distribution vault, breaking isolation between separate reward campaigns.

Remediation

Utilize an `is_closed` flag to prevent reuse of the same distribution identity, rather than deleting the distribution PDA outright.

Patch

Resolved in PR#32.

Underfunded Allocations Due to Transfer-Fee Mints MEDIUM OS-SPR-ADV-02

Description

`AddDirectRecipient` increases the recipient's recorded allocation and the distribution's `total_allocated` by the full amount in instruction data. But it funds the vault utilizing a token `TransferChecked`, which for fee-on-transfer mints may credit the vault with less than amount because part of the transfer is deducted as a transfer fee. This creates an accounting mismatch as on-chain reward state assumes the full allocation is funded, while the vault actually holds fewer tokens.

Remediation

Measure the vault's actual post-transfer balance delta and record only the net amount received.

Patch

Resolved in PR#34.

Parent Closure Breaks Recipient Account Cleanup LOW OS-SPR-ADV-03

Description

There is a potential for a deadlock during cleanup between `CloseDirectDistribution` and `CloseDirectRecipient`. `CloseDirectRecipient` requires the provided distribution account to still be owned by the rewards program, because the account parser enforces `verify_current_program_account(distribution)`.

>_ *program/src/instructions/direct/close_recipient/accounts.rs* RUST

```rust
impl<'a> TryFrom<&'a [AccountView]> for CloseDirectRecipientAccounts<'a> {
    type Error = ProgramError;

    #[inline(always)]
    fn try_from(accounts: &'a [AccountView]) -> Result<Self, Self::Error> {
        let [recipient, original_payer, distribution, recipient_account, event_authority,
            ↪ program] = accounts else {
            return Err(ProgramError::NotEnoughAccountKeys);
        };
        [...]
        // 4. Validate accounts owned by current program
        verify_current_program_account(distribution)?;
        verify_current_program_account(recipient_account)?;

        Ok(Self { recipient, original_payer, distribution, recipient_account, event_authority,
            ↪ program })
    }
}
```

However, as `CloseDirectDistribution` deletes the distribution PDA, this check will never pass for any remaining recipient accounts. As a result, `DirectRecipient` accounts that were not closed before become permanently unclosable. This strands the rent in the recipient PDAs and leaves a stale state on-chain indefinitely.

Remediation

Allow `CloseDirectRecipient` to succeed for fully claimed recipient accounts even after the parent distribution PDA has been closed.

Patch

Resolved in PR#32.

Misleading Anchor-Compatible Event Encoding LOW OS-SPR-ADV-04

Description

The rewards program emits CPI events with Anchor's event prefix (`EVENT_IX_TAG_LE`) but does not follow Anchor's full event format. Instead of Anchor's standard 8-byte event discriminator (`sha256("event:<EventStruct>")[..8]`), it utilizes only a 1-byte custom discriminator. As a result, the events appear Anchor-compatible while being structurally incompatible with standard Anchor decoders.

>_ *program/src/traits/event.rs* RUST

```rust
/// Event discriminator with Anchor-compatible prefix
pub trait EventDiscriminator {
    /// Event discriminator byte
    const DISCRIMINATOR: u8;

    /// Full discriminator bytes including EVENT_IX_TAG_LE prefix
    #[inline(always)]
    fn discriminator_bytes() -> Vec<u8> {
        let mut data = Vec::with_capacity(EVENT_DISCRIMINATOR_LEN);
        data.extend_from_slice(EVENT_IX_TAG_LE);
        data.push(Self::DISCRIMINATOR);
        data
    }
}
```

Remediation

Utilize Anchor's full 8-byte event discriminator (`sha256("event:<EventStruct>")[..8]`) instead of a 1-byte custom tag.

Patch

Resolved in PR#37.

Recipient-Only Closure Creates Incentive Mismatch LOW OS-SPR-ADV-05

Description

The `CloseDirectRecipient` instruction in direct distribution requires the `recipient` to sign, but the lamports recovered from closing the `recipient_account` are sent to `original_payer`. That implies the only party authorized to trigger cleanup is not the party that benefits from it. Thus, once a recipient has fully claimed their allocation, they lack economic incentive to incur an additional transaction fee solely to return rent to another party. Consequently, fully claimed recipient PDAs may remain open indefinitely.

Remediation

Make closure permissionless once `claimed_amount == total_amount`, while still directing rent to the stored payer, or allow the `original_payer` itself to initiate closure.

Patch

Resolved in PR#35.

05 — General Findings

Here, we present a discussion of general findings identified during our audit. While these findings do not pose an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-SPR-SUG-00	In `RevokeMerkleClaim`, `NonVested` mode may pay the claimant their vested-but-unclaimed tokens without updating `MerkleClaim.claimed_amount` to reflect that payout.
OS-SPR-SUG-01	Several processors parse instruction data but omit `ix.data.validate()`, which is currently harmless but introduces latent risk if validation rules are added in the future.

Failure to Update Merkle Claim State OS-SPR-SUG-00

Description

In `RevokeMerkleClaim`, a `NonVested` revocation may transfer the claimant's vested but unclaimed amount without updating `MerkleClaim.claimed_amount`, resulting in inconsistent accounting where tokens are paid out but still recorded as unclaimed.

Remediation

Update `claimed_amount` to include any vested amount paid during `NonVested` revocation before finalizing the revocation.

Patch

Resolved in PR#32.

Missing Instruction Data Validation Hooks OS-SPR-SUG-01

Description

Several processors, including `ClaimDirect`, `ClaimMerkle`, and the close instructions in Direct and Merkle distributions (`CloseDirectDistribution`, `CloseDirectRecipient`, `CloseMerkleClaim`, and `CloseMerkleDistribution`), parse instruction data but do not call `ix.data.validate()` before execution. Currently, this is mostly benign because some of these data structures are empty or have no extra validation rules, but it creates a forward-compatibility risk. If validation logic is later added to any of these instruction-data types, these processors will silently bypass it while other instructions enforce theirs correctly.

Remediation

Call `ix.data.validate()?` uniformly in every processor after parsing.

Patch

Resolved in PR#36.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.